

Reinforcement Learning framework for Optimal Trade Execution

Andy Au

April 2026

1 The Problem

A large institutional investor needs to liquidate Q shares of an asset over a fixed horizon $[0, T]$. Selling too fast moves the price against you; selling too slow exposes you to volatility risk. The goal is to find a trading schedule that minimizes total execution cost.

Let q_t denote remaining inventory at time t , with $q_0 = Q$ and $q_T = 0$. The midprice S_t follows:

$$dS_t = \mu dt + \sigma dW_t - g(v_t) dt,$$

- $v_t = -\dot{q}_t$: selling rate (units per time).
- $g(v_t)$: **permanent impact**: each unit sold pushes the midprice down permanently.
- $h(v_t)$: **temporary impact**: the cost of accepting a price below midprice to sell immediately. (Does not appear in dS_t but enters the execution cost.)

The execution cost is

$$\text{Cost} = \underbrace{\int_0^T v_t [h(v_t) + g(v_t)] dt}_{\text{impact cost}} + \underbrace{\lambda \int_0^T \sigma^2 q_t^2 dt}_{\text{inventory penalty}},$$

- **Impact cost**: total price paid from both permanent and temporary impact while selling.
- **Inventory penalty**: penalizes holding unsold shares, which are exposed to price volatility σ^2 . Controlled by risk-aversion λ .

Why this matters

If you want to liquidate a \$100M position, the difference between selling everything at market open, and an optimal one can easily be 50–100 bps (\$500K–\$1M). At scale, execution efficiency is a direct driver of fund returns.

2 Classical Solutions

Almgren–Chriss (2001)

The standard model assumes linear impact:

- $g(v) = \gamma v$: **permanent impact**: γ measures how much each unit of selling permanently moves the midprice.
- $h(v) = \eta v$: **temporary impact**: η measures the extra cost per share for trading at rate v (book depth, spread).

Under these assumptions the problem is a deterministic linear-quadratic control, and the optimal trading rate has a closed-form solution:

$$v_t^* = \frac{\kappa}{\sinh(\kappa T)} \cosh(\kappa(T - t)) \cdot Q, \quad \kappa = \sqrt{\frac{\lambda \sigma^2}{\eta}}.$$

- **Shape**: U-shaped: sell fast early (inventory risk is highest), slow in the middle (let the book recover), fast again near T (hard deadline forces completion).
- κ (**urgency**): ratio of inventory pain ($\lambda \sigma^2$) to trading pain (η). Large κ means risk dominates cost, so sell aggressively. Small κ means impact dominates risk, so spread trades evenly. The solution flattens toward TWAP ($v_t = Q/T$).

Limitations

The Almgren–Chriss solution is clean but relies on strong assumptions:

1. **Linear impact**. Empirical evidence suggests impact is concave (roughly $\sqrt{v}$), meaning the first units are more expensive per unit than later ones. Linear models overestimate impact at low rates and underestimate at high rates.
2. **Static parameters**. Volatility, spread, and liquidity all vary intraday and across market regimes. The AC solution uses a single σ , η , γ for the entire horizon.
3. **No adaptation**. The optimal schedule is computed at $t = 0$ and executed open-loop. It cannot react to favorable price moves, sudden liquidity, or adverse drift.
4. **No microstructure**. Real execution involves discrete order books, minimum tick sizes, queue priority, and fill uncertainty. None of this appears in the continuous diffusion model.

More sophisticated classical approaches (Obizhaeva–Wang 2013 with transient impact, Cartea–Jaimungal with ambiguity aversion) address some of these, but each closed-form extension adds one feature at a time. The interaction of multiple real-world frictions simultaneously is analytically intractable.

3 Why Reinforcement Learning

RL is a natural fit for optimal execution because the problem has the right structure: sequential decisions, a clear objective (minimize cost), and a rich state that is hard to summarize in a closed-form control law.

What RL adds over classical methods

1. **Nonlinear impact models.** RL does not require the impact function to be linear or even parametric. The agent learns the cost structure from simulated or historical fills. This lets us handle concave permanent impact, nonlinear temporary impact, or even impact that depends on recent trading history.
2. **Adaptive execution.** The RL agent observes the current market state and adjusts its trading rate accordingly. If volatility spikes, it can accelerate; if a favorable price move occurs, it can slow down. This is closed-loop control, which the AC solution is not.
3. **Rich state representation.** Beyond (t, q_t, S_t) , we can feed the agent order book depth, recent volume, realized volatility, spread, momentum signals, or time-of-day features. Classical models cannot incorporate these without becoming analytically intractable; RL handles high-dimensional states naturally.
4. **Realistic constraints.** Minimum lot sizes, maximum participation rates, auction windows, and other operational constraints can be encoded directly in the action space or as penalties in the reward. No need to re-derive the solution each time a constraint changes.

Why RL is not a black box here

A common objection to RL in finance is interpretability. But optimal execution has strong structural conditions that constrain the problem:

- The action space is low-dimensional (how much to trade in each period).
- The Almgren–Chriss solution provides a known baseline — the RL agent should match or beat it, and we can find exactly where and why it deviates.
- The reward (negative execution cost) is well-defined and non-gameable.
- We can verify the learned policy against known solutions in the linear-impact regime as a sanity check.

4 Proposed Approach

We start simple: start by reproducing the Almgren–Chriss solution in RL terms. If the agent can match the known optimal in the linear regime, we have a validated baseline. Then we introduce the real-world frictions that AC cannot handle and let the agent adapt.

Phase 1: Match AC

Environment

We discretize $[0, T]$ into N steps of size $\Delta t = T/N$. At each step k :

1. Agent observes state s_k and selects action a_k (shares to sell).
2. Execution occurs: temporary and permanent impact applied, inventory updated.
3. Price evolves: $S_{k+1} = S_k + \mu\Delta t + \sigma\sqrt{\Delta t}\epsilon_k - \gamma a_k$, where $\epsilon_k \sim \mathcal{N}(0, 1)$.
4. Agent receives reward r_k .

Episode ends at step N , with a penalty if $q_N > 0$ (incomplete liquidation).

State

Start minimal by using only the information that the AC solution depends on:

$$s_k = \left(\frac{k}{N}, \frac{q_k}{Q}, \frac{S_k}{S_0} \right),$$

- k/N : normalized time (how far into the horizon).
- q_k/Q : inventory fraction remaining.
- S_k/S_0 : relative price level.

Normalization keeps inputs in $[0, 1]$, which stabilizes training. In this minimal state, the RL agent has access to exactly the same information as AC.

Action

$a_k = f_k \cdot q_k$, where $f_k \in [0, 1]$ is the fraction of remaining inventory to sell in period k . This way the action space is the same regardless of order size.

Reward

The reward mirrors the AC cost function directly:

$$r_k = - \left(\underbrace{a_k \cdot \eta \cdot a_k / \Delta t}_{\text{temporary impact cost}} + \underbrace{a_k \cdot \gamma \cdot a_k}_{\text{permanent impact cost}} + \underbrace{\lambda \sigma^2 q_k^2 \Delta t}_{\text{inventory penalty}} \right).$$

The agent is optimizing the same thing AC optimizes, so any performance difference is purely due to the policy, not a different objective. A terminal penalty $-M \cdot q_N^2$ forces full liquidation.

Why PPO

- **Continuous actions:** the selling rate is continuous.
- **Stability:** PPO’s clipped surrogate objective prevents large policy updates, which matters here because the reward signal is noisy (driven by price randomness).
- **Gaussian policy:** PPO outputs a mean and variance for the action. In the linear-impact regime, we expect the learned policy to converge toward the AC deterministic solution (variance $\rightarrow 0$). Monitoring this convergence will be useful.

Validation

In the linear regime with known $(\gamma, \eta, \sigma, \lambda)$, the RL agent should:

- Match the AC trading schedule v_t^* (compare learned policy mean to analytical solution).
- Achieve the same expected cost (within noise).
- Converge to near-zero policy variance (deterministic control is optimal here).

If it does not, the issue is in training, not in the approach. This is the sanity check before moving on.

Phase 2: Extend Beyond AC

Once the agent matches AC in the linear regime, we introduce frictions that AC cannot handle. Each extension targets a specific limitation identified in Section 2.

Nonlinear impact

Replace $g(v) = \gamma v$ with $g(v) = \gamma\sqrt{v}$ (concave, empirically motivated). The RL agent should learn a different schedule; less aggressive at low rates, more aggressive at high rates, without any change to the algorithm.

Time-varying volatility

Make σ a function of time-of-day or recent realized volatility. This requires expanding the state:

$$s_k = \left(\frac{k}{N}, \frac{q_k}{Q}, \frac{S_k}{S_0}, \hat{\sigma}_k \right),$$

where $\hat{\sigma}_k$ is a rolling realized volatility estimate. The agent can now learn to sell faster during high volatility periods and slow down when volatility drops.

Further state features

Additional features are added only when there is a clear hypothesis for why they help:

- **Spread:** wider spread $\Rightarrow$ higher temporary impact $\Rightarrow$ agent should slow down.
- **Volume:** high market volume $\Rightarrow$ more liquidity $\Rightarrow$ agent can sell faster with less impact.
- **Momentum:** recent price trend $\Rightarrow$ if price is falling, urgency increases.

We should not add features blindly. Each addition has a testable prediction: if adding spread to the state does not change the learned policy, it is not informative and should be removed. If dimensionality grows, PCA or KLE compression can extract the relevant structure.

Diagnostics and Iteration

We log the following throughout training to guide model development:

- **Implementation shortfall vs AC:** are we beating the classical baseline? By how much, and in which market conditions?
- **Policy entropy:** should decrease over training as the agent becomes more certain. If entropy plateaus high, the agent is not learning a clear strategy.
- **Action distribution vs AC schedule:** where in the trajectory does the RL agent deviate? Early? Late? During high-volatility periods? This tells us what the agent is exploiting.
- **KL divergence between policy updates:** measures training stability. Spikes indicate the agent is making erratic updates. We may need to reduce learning rate or increase clipping.
- **Episode cost variance:** low variance = consistent execution. High variance may indicate the agent is taking unnecessary risks.
- **Inventory trajectory shape:** plot q_t over time. Sensible policies should be monotonically decreasing and roughly follow the AC shape. Pathological behavior (holding everything until the last step) is immediately visible.

The iteration logic is: if the agent stagnates or underperforms, the diagnostics tell us whether the issue is reward shaping (change the objective), missing features (expand the state), or training dynamics (adjust hyperparameters). We shouldn't guess.